GUIDE 2 OF 8 · AI LEARNING SERIES

MCP — Model Context Protocol

Connect AI models to any tool, data source, or service — standardized

01 What is MCP?

MCP (Model Context Protocol) is an open standard (released by Anthropic, Nov 2024) that defines how AI models — like Claude or GPT-4 — connect to external tools, databases, APIs, and file systems in a **standardized, secure, and pluggable** way.

Before MCP: Every AI assistant had its own custom way to call tools. Each integration was one-off and brittle. **After MCP:** Any MCP-compatible client (Claude Desktop, Cursor, VS Code Cline) can connect to any MCP-compatible server (GitHub, databases, file systems, APIs) without custom glue code.

> **SIMPLE ANALOGY**
>
> MCP is like USB-C for AI tools. Just as USB-C lets you plug any device into any laptop, MCP lets any AI model connect to any tool server using the same protocol.

02 MCP Architecture

MCP has three roles:

Role	What It Is	Examples
MCP Host	The AI application that wants to use tools	Claude Desktop, Cursor IDE, VS Code + Cline, custom app
MCP Client	Protocol layer inside the host that manages connections	Built into Claude Desktop, Cline extension
MCP Server	A small server exposing tools/resources via MCP protocol	GitHub MCP, Postgres MCP, filesystem MCP, your custom server

Communication flow: Host (Claude) ↔ MCP Client ↔ [stdio or HTTP/SSE] ↔ MCP Server ↔ External Tool/DB/API

03 Three Core MCP Primitives

1. Tools — Functions the model can call

Tools are functions exposed by an MCP server that the AI can invoke. Like function calling but standardized. The model decides when to call a tool, what arguments to pass, and processes the result.

```
# Tool definition in MCP server (Python SDK)
```

```
from mcp.server import Server
from mcp.server.stdio import stdio_server
from mcp import types

app = Server("my-tool-server")

@app.list_tools()
async def list_tools() -> list[types.Tool]:
    return [types.Tool(
        name="search_database",
        description="Search product database by keyword",
        inputSchema={"type": "object",
            "properties": {"query": {"type": "string"}},
            "required": ["query"]}
    )]

@app.call_tool()
async def call_tool(name: str, arguments: dict):
    if name == "search_database":
        results = db.search(arguments["query"])
        return [types.TextContent(type="text", text=str(results))]
```

2. Resources — Data the model can read

Resources are read-only data sources exposed by the server. The model can list available resources and read their content — like files, database records, API responses.

```
# Resource definition
@app.list_resources()
async def list_resources():
    return [types.Resource(
        uri="file:///data/config.json",
        name="App Configuration",
        mimeType="application/json"
    )]

@app.read_resource()
async def read_resource(uri: str):
    content = open(uri.replace("file://", "")).read()
    return types.ReadResourceResult(
        contents=[types.TextResourceContents(uri=uri, text=content)])
```

3. Prompts — Reusable prompt templates

Prompt templates defined by the server that the AI host can discover and use. Allows server authors to embed expert prompts alongside their tools.

04 Setting Up MCP — Step by Step

Install the Python SDK

```
# Installation
pip install mcp
# or using uv (recommended)
uv add mcp
```

Run a Pre-built MCP Server (e.g. filesystem)

```
# Claude Desktop config (~/.claude/claude_desktop_config.json)
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem",
               "/Users/you/Documents"]
    },
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {"GITHUB_PERSONAL_ACCESS_TOKEN": "ghp_..."}
    }
  }
}
```

Build a Custom MCP Server

```
# Complete minimal MCP server
import asyncio
from mcp.server import Server
from mcp.server.stdio import stdio_server
from mcp import types

app = Server("my-server")

@app.list_tools()
async def list_tools():
    return [types.Tool(name="add_numbers",
        description="Adds two numbers",
        inputSchema={"type":"object",
            "properties":{"a":{"type":"number"},"b":{"type":"number"}},
            "required":["a","b"]})]

@app.call_tool()
async def call_tool(name, args):
```

```
    if name == "add_numbers":
        result = args["a"] + args["b"]
        return [types.TextContent(type="text", text=str(result))]

async def main():
    async with stdio_server() as (r, w):
        await app.run(r, w, app.create_initialization_options())

if __name__ == "__main__":
    asyncio.run(main())
```

05 Popular Pre-built MCP Servers

Server	What It Exposes	Install
filesystem	Read/write local files and directories	npx @modelcontextprotocol/server-filesystem
github	Repos, issues, PRs, file contents	npx @modelcontextprotocol/server-github
postgres	Query PostgreSQL databases	npx @modelcontextprotocol/server-postgres
brave-search	Web search via Brave API	npx @modelcontextprotocol/server-brave-search
slack	Read channels, send messages	npx @modelcontextprotocol/server-slack
memory	Persistent key-value memory for the model	npx @modelcontextprotocol/server-memory
puppeteer	Browser automation, screenshots	npx @modelcontextprotocol/server-puppeteer
sqlite	Query SQLite databases	npx @modelcontextprotocol/server-sqlite
Custom Python	Your own tools and data sources	pip install mcp + write your server

06 MCP vs Function Calling vs Plugins

Approach	Scope	Reusability	Standardized?
Function Calling (OpenAI)	Single API call, defined per-request	None — redefine every call	Per-provider only
ChatGPT Plugins (deprecated)	Per-app plugins, OpenAI-specific	Within ChatGPT only	OpenAI-specific
MCP	Persistent servers, any host, any model	Full — one server, many hosts	Yes — open standard

KEY ADVANTAGE

Write your MCP server ONCE. Use it from Claude Desktop, from Cursor, from your custom app, from any MCP-compatible host — without any changes to the server.

07 MCP Tools Ecosystem

Tool	Purpose
Claude Desktop	Primary MCP host — configure servers in claude_desktop_config.json
Cursor IDE	AI coding IDE with native MCP support
VS Code + Cline	Open-source coding agent with MCP support
MCP Python SDK	pip install mcp — build Python MCP servers
MCP TypeScript SDK	npm install @modelcontextprotocol/sdk — build Node.js servers
MCP Inspector	Debug and test MCP servers — npx @modelcontextprotocol/inspector
uv / uvx	Fast Python package manager — recommended for MCP server installs
MCP Hub (glama.ai)	Directory of community MCP servers